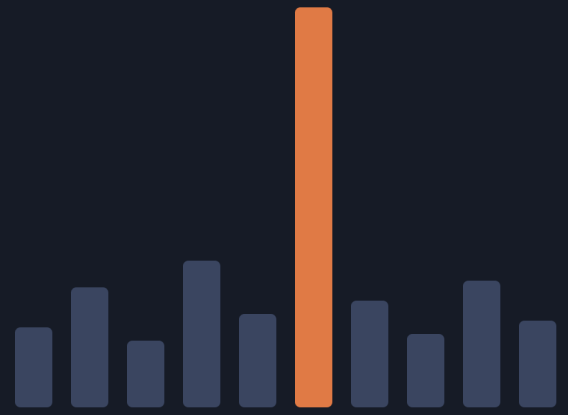
SEMANA 6 · QUANTIZAÇÃO

Outliers e granularidade

Por que quantizar ativação é mais difícil que quantizar peso?



Quantizar: floats viram poucos inteiros

A receita (INT8 simétrico)

$$\Delta = \max|x| \,/\, 127$$

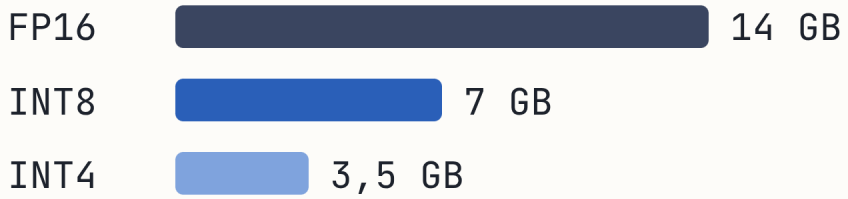
$$q = \mathrm{round}(x \,/\, \Delta)$$

$$\hat{x} = q \cdot \Delta$$

```
x = [0.42, -1.27, 0.80]
Δ = 1.27 / 127 = 0.01
q = [42, -127, 80]
```

Δ é o tamanho do degrau: quanto maior, mais grosseira fica a régua.

Modelo de 7B parâmetros



Pesos (W): fixos — quantiza uma vez, offline.

Ativações (X): mudam a cada entrada — é aqui que mora o problema.

```
W8A8 = pesos e ativações em 8 bits
W4A16 = só os pesos, em 4 bits
```

Um único outlier esmaga todos os outros valores

x	q sem o 100	q com o 100	x̂ com o 100
0.3	42	0	0.00
-0.7	-99	-1	-0.79
0.5	71	1	0.79
100	—	127	100
-0.2	-28	0	0.00
0.9	127	1	0.79
-0.4	-56	-1	-0.79
0.1	14	0	0.00

Δ: 0,007 → 0,787

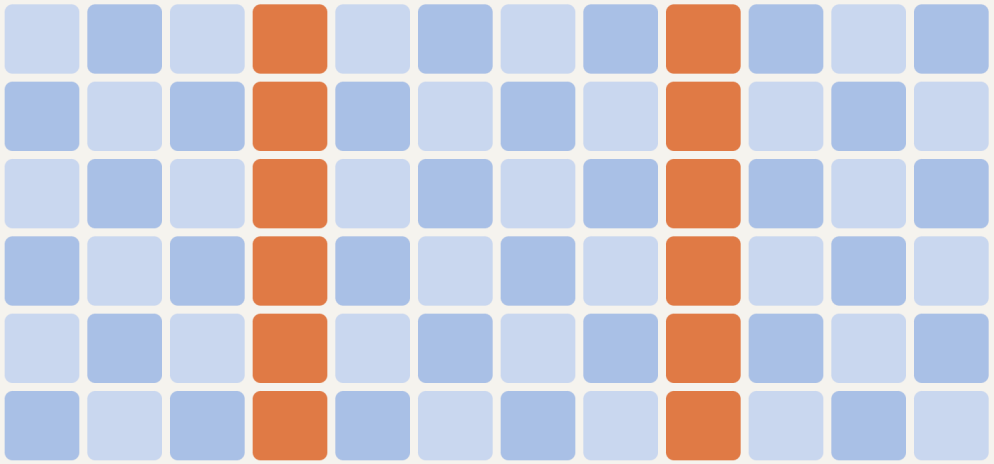
3 de 255

níveis sobram para os 7 valores normais: −0,79, 0 e +0,79.

Tudo entre −0,39 e +0,39 vira exatamente zero: a informação some.

Os outliers moram nas ativações, sempre nos mesmos canais

canais (features) → · cada linha é um token



Se um canal tem outlier, ele aparece em **todos os tokens**. Já os pesos têm distribuição plana: são fáceis de quantizar.

~100×
maiores que o resto das ativações

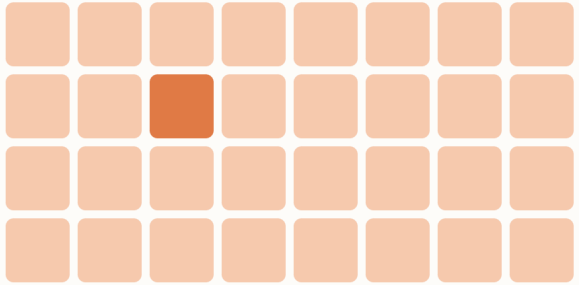
6 canais
concentram ~150 mil outliers por sequência num modelo de 6,7B

+600–1000%
de perplexidade ao zerar só esses canais (zerar canais aleatórios: ~0,1%)

Fontes: LLM.int8() (Dettmers et al., 2022), seção 4 · SmoothQuant (Xiao et al., 2023), seção 3

Granularidade: quantos Δ por matriz

Por tensor

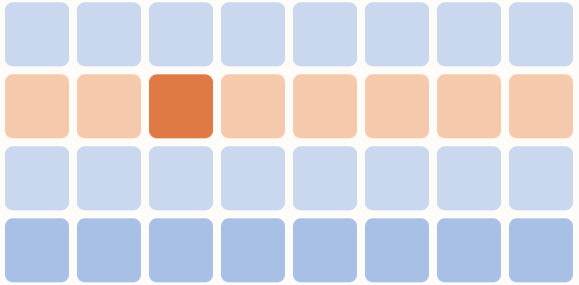


1 Δ para a matriz inteira

✓ 1 número só; o mais rápido

✗ um outlier define o Δ de todos

Por canal

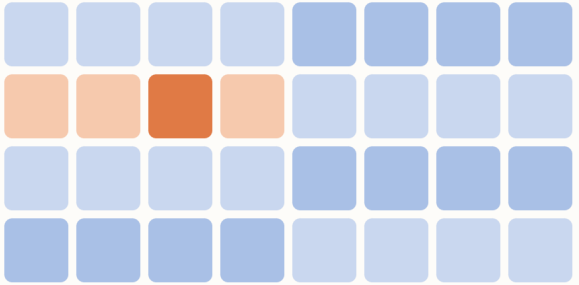


1 Δ por linha (canal ou token)

✓ isola o estrago; roda no kernel INT8

✗ só ajuda se a escala seguir a direção do outlier

Por grupo



1 Δ a cada g valores (g = 32–128)

✓ o mais preciso; padrão em 4 bits

✗ ≈ 0,15 bit extra por peso e kernel próprio

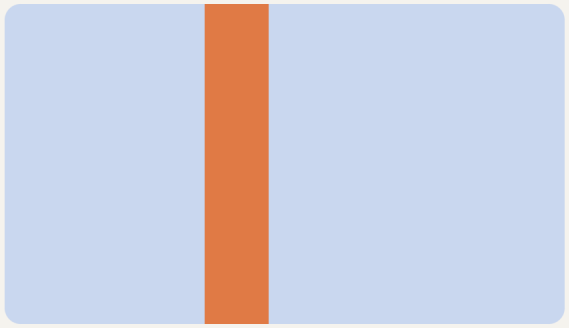
outlier

dividem o Δ com ele, logo são esmagados

cada tom = um Δ diferente

A pegadinha: no matmul INT8, a escala só pode ficar por fora

X (tokens × canais)



×

W (canais × saídas)



✓ Δ por **linha de X** (por token) e por **coluna de W**
✗ Δ por **coluna de X** (por canal): é a dimensão que o matmul soma

$$Y = \mathrm{diag}(\Delta_X) \cdot (\bar{X} \cdot \bar{W}) \cdot \mathrm{diag}(\Delta_W)$$

OPT-175B, INT8	acurácia
FP16 (referência)	71,6%
por tensor	32,3%
por token	31,7%
por canal*	71,4%

* simulado: resolveria, mas não roda no kernel INT8.

Fonte: SmoothQuant (Xiao et al., 2023), Tabela 1 e Eq. 2 — média em WinoGrande, HellaSwag, PIQA e LAMBADA

Então: por que ativação é mais difícil que peso?

1. Ela muda a cada entrada

O peso é conhecido antes; a ativação só existe quando o texto chega. Para fixar o Δ é preciso estimá-lo com dados de calibração ou calculá-lo na hora.

2. Ela tem outliers enormes

Valores ~100× maiores que o resto, e um só já esmaga a régua inteira. Os pesos têm distribuição plana.

3. Os outliers moram nos canais

Sempre nas mesmas colunas de X, em todos os tokens — não dá para isolá-los por token.

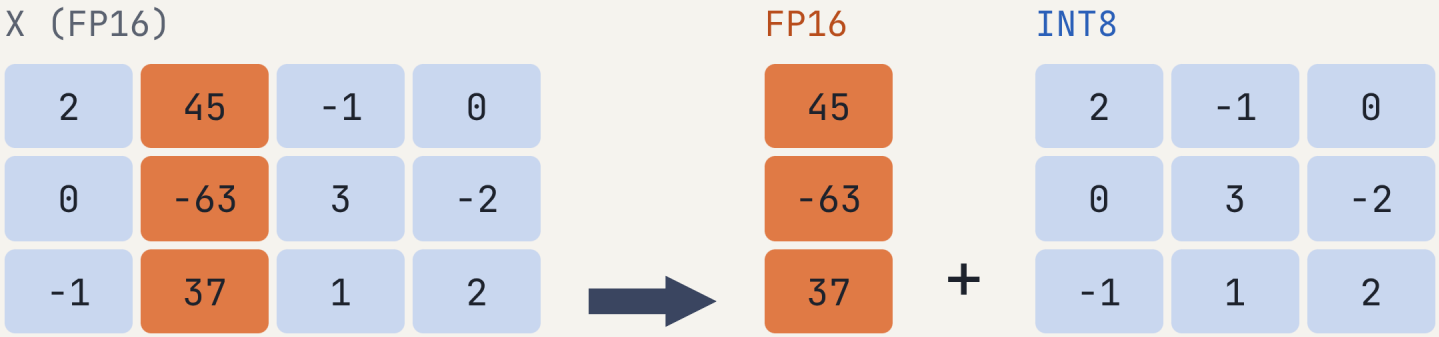
4. O hardware não escala por canal

No matmul INT8, a escala de X só pode ser por token. A granularidade que resolveria não roda.

Por isso há dois caminhos: tratar o outlier na ativação (W8A8) ou deixar a ativação em FP16 e quantizar só o peso (W4A16).

IDEIA 1 · ISOLAR

LLM.int8(): colunas-outlier ficam em FP16



Como funciona

- Coluna com algum |valor| ≥ 6 vai para um matmul em FP16
- Só ≈ 0,1% das dimensões; os outros 99,9% ficam em INT8
- No resto: um Δ por linha de X e por coluna de W

Na prática

- ✓ Sem perda de qualidade até 175B parâmetros
- ✓ Memória cai quase pela metade
- ✗ Dois matmuls + separar e juntar: muitas vezes mais lento que FP16

IDEIA 2 · MOVER

SmoothQuant: move o problema para o peso

$$X \cdot W = (X \div s) \cdot (s \cdot W)$$

Mesma saída: s divide cada canal da ativação e multiplica a linha correspondente do peso.

Exemplo (α = 0,5)

```
max|X| por canal = [2, 16, 2, 9]
max|W| por linha = [2, 1, 2, 1]
s = √(max|X| ÷ max|W|) = [1, 4, 1, 3]
canal 2 de X: [-16, 8] ÷ 4 → [-4, 2]
linha 2 de W: [1, -1, -1] × 4 → [4, -4, -4]
maior |valor|: X 16 → 4 · W 2 → 4
```

Por que funciona bem

- s é calculado offline (~512 frases de calibração) e embutido na camada anterior: custo zero em execução
- α decide quanto da dificuldade vai para o peso; 0,5 costuma ser o ponto ideal
- Tudo em INT8 de verdade (W8A8): até 1,56× mais rápido e ~2× menos memória

IDEIA 3 · COMPENSAR

GPTQ: compensa o erro nos pesos vizinhos

Só pesos. Quantiza uma coluna por vez e corrige as que ainda faltam para manter a saída da camada.

Exemplo (grade de inteiros)

$y = w_1 \cdot x_1 + w_2 \cdot x_2$ e $x_1 \approx x_2$

$w = [0.6, 0.6] \rightarrow y \approx 1.2 \cdot x$

RTN: $[1, 1] \rightarrow y \approx 2.0 \cdot x$ (erro 0.8)

GPTQ: $w_1 \rightarrow 1$, sobra erro +0.4

$w_2 \leftarrow 0.6 - 0.4 = 0.2 \rightarrow 0$

$[1, 0] \rightarrow y \approx 1.0 \cdot x$ (erro 0.2)

O que ele precisa

- Saber quais entradas andam juntas: $H = 2XX^T$, calculada com os dados de calibração
- Ordem fixa de colunas e atualização em blocos de 128: 175B em ~4 h numa GPU
- Combina com grupos (g = 128) e até com o AWQ

IDEIA 4 · PROTEGER

AWQ: protege os pesos mais importantes

Só os pesos são quantizados (4 ou 3 bits). A pergunta: quais 1% dos pesos importam de verdade?

OPT-6.7B, INT3 (g = 128)	PPL ↓
FP16, sem quantizar	10,86
INT3 ingênuo (RTN)	23,54
1% em FP16, escolhido pelo peso	22,37
1% em FP16, escolhido pela ativação	11,39
AWQ: tudo INT3 + escala	11,39

O truque: escalar em vez de manter em FP16

$$Q(w \cdot s) \cdot (x \div s)$$

Multiplica o canal importante por s antes de quantizar e divide a entrada por s: o erro relativo desse peso cai ≈ 1/s.

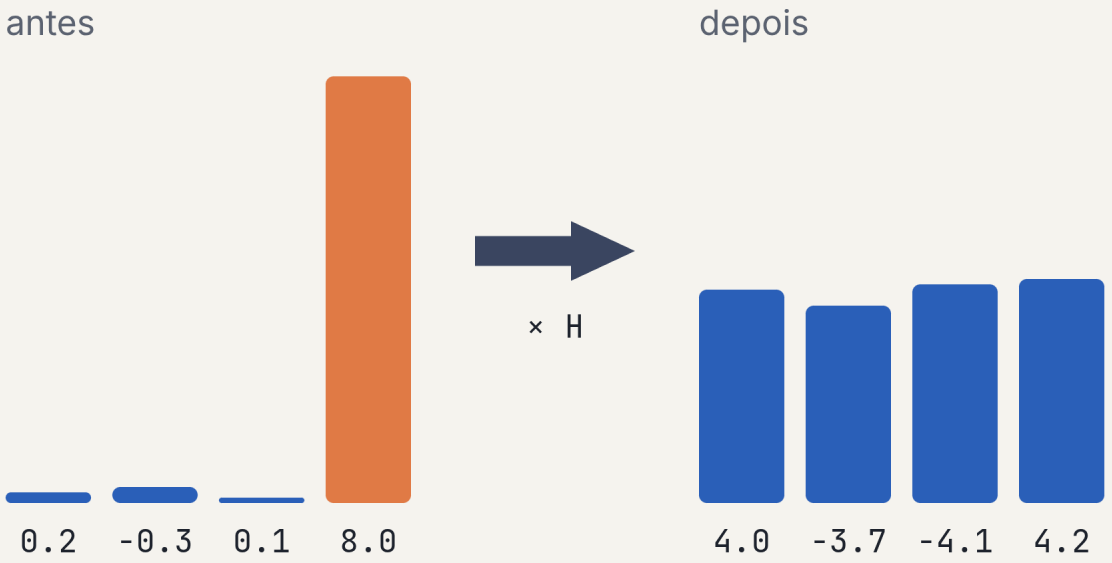
s vem da magnitude média da ativação; só um expoente α é ajustado, por busca em grade.

Sem backprop nem reconstrução: não decora os dados de calibração.

Fonte: AWQ (Lin et al., 2024), Tabelas 1 e 3 — perplexidade no WikiText-2

FRONTEIRA · ROTACIONAR

QuaRot e SpinQuant: rotacionar



R é ortogonal ($R \cdot R^T = I$), então $X \cdot W = (X \cdot R) \cdot (R^T \cdot W)$: **mesma saída**.

O outlier se espalha por todos os canais: maior/menor |valor| cai de 80× para ~1,1×.

QuaRot: rotação de Hadamard fixa → 4 bits até nas ativações e no KV cache.

SpinQuant: aprende a rotação em vez de fixá-la.

O que cada método assume

Método	Formato	Assume que...	Quando a premissa falha...
LLM.int8()	W8A8	os outliers são poucos (≤ 7 dimensões) e um limiar fixo ($\|x\| \geq 6$) os acha na hora	com muitos outliers, sobra FP16 demais e a economia some
SmoothQuant	W8A8	os outliers ficam nos mesmos canais para qualquer texto, e o peso aguenta parte da dificuldade	modelos mais difíceis pedem α maior (GLM-130B usou 0,75)
GPTQ	W4A16	o erro de um peso pode ser compensado pelos outros, e a calibração representa o uso real	sobreajuste: com calibração de outro domínio, a PPL piora 2,3–4,9
AWQ	W4A16	peso importante é o que recebe ativação grande; escalar um canal quase não muda o Δ do grupo	s grande demais prejudica os outros canais (s = 4 pior que s = 2)

Fontes: LLM.int8() seç. 3.2 · SmoothQuant seç. 4 e 5.1 · AWQ Tab. 2 e Fig. 8

Onde entram os dados de calibração

Calibração = um pouco de texto genérico passado pelo modelo só para observar as ativações. Sem treino.

Método	Dados	Para quê
LLM.int8()	nenhum	tudo em execução (dinâmico): o limiar acha os outliers a cada entrada
SmoothQuant	512 frases do Pile	max|X| por canal → s; no modo estático, também os Δ das ativações
GPTQ	128 trechos de 2048 tokens do C4	$H = 2XX^T$ de cada camada, para saber como compensar o erro
AWQ	pequeno conjunto do Pile (~16 sequências já bastam)	magnitude média por canal → s, e a busca do α

Estática × dinâmica: com calibração, o Δ da ativação é fixado antes (mais rápido); sem ela, é calculado a cada execução. **Risco:** quanto mais o método usa os dados, mais se ajusta a eles — fora do domínio, o GPTQ piora 2,3–4,9 de PPL; o AWQ, só 0,5–0,6.

Para SLMs: pesos por grupo, ativações por token

```
pesos → por grupo (g = 32–128) · ativações → por token
```

Modelo pequeno sofre mais

Menos redundância: cada erro pesa mais. OPT-1.3B em 4 bits ingênuo: PPL 14,6 → 48,2.

Grupo custa pouco

g = 128 adiciona ≈ 0,15 bit por peso: 4 bits viram ~4,15. E o outlier só estraga o próprio bloco.

O gargalo é memória

No dispositivo, gerar texto é limitado por ler pesos. Por isso o padrão é só-pesos por grupo: AWQ, GPTQ, llama.cpp.

Ressalva: o LLM.int8() mostra que os outliers acompanham a qualidade do modelo (perplexidade), não só o tamanho — um SLM bem treinado também pode tê-los.